



מדעי המחשב

בחינת מתכונת נוספת - תש"פ

משך הבחינה : 3 שעות.

מבנה השאלון ומפתח ההערכה : בשאלון זה שלושה פרקים

פרק ראשון – בפרק זה שלוש שאלות. יש לענות על השאלה הראשונה ולבחור אחת מבין שתי השאלות האחרות.

$(10X1) + (15X1)$ -- 25 נקודות

פרק שני – בפרק זה שלוש שאלות, ומהן עליך לענות על שתיים.

$(25X2)$ -- 50 נקודות

פרק שלישי – בשאלון הבגרות פרק זה מכיל שאלות בארבעה מסלולים שונים. ענה על שאלה אחת במסלול שלמדת. בשאלון הנוכחי שתי שאלות במסלול "תכנות מונחה עצמים", מהן עליך לענות על שאלה אחת.

$(25X1)$ -- 25 נקודות

סה"כ -- 100 נקודות

חומר עזר מותר בשימוש : כל חומר עזר חוץ ממחשב הניתן לתכנות.

נא לשמור על טוהר הבחינות.

שם : _____ כיתה : _____ מורה : _____

פרק		ראשון			שני		שלישי	
שאלה	1	2	3	4	5	6	7	8
בחרתי ב-	✓							

ההנחיות בשאלון זה מנוסחות בלשון זכר ומכוונות לנבחנות ולנבחנים כאחד.

בהצלחה !

פרק ראשון (25 נקודות)

בפרק זה עליך לענות על שאלה 1 ובנוסף על אחת מבין השאלות 2 ו-3

שאלה 1 – חובה – 10 נקודות

כתוב פעולה חיצונית **CountK** המקבלת כפרמטרים את arr - מערך שלמים ואת k - שלם. הפעולה תמנה ותחזיר כמה זוגות של ערכים סמוכים במערך arr סכומם הוא k. למשל, עבור המערך הבא ו-k=100 הפעולה תחזיר 3 כי ישנם שלושה זוגות סמוכים שסכומם הוא 100.

0	1	2	3	4	5	6	תא
16	84	9	27	73	27	420	ערך

שאלה 2 – 15 נקודות

בשל מגיפת הקורונה, בית הספר מחלק את תלמידי שכבת י"א לקפסולות, כשבכל קפסולה 20 תלמידים. "קפסולה" (Capsule) מאופיינת על ידי מספרי הזהות של תלמידים (students, מערך מחרוזות בגודל 20).

במחלקה "קפסולה" נתונה הפעולה **InCapsule** המקבלת כפרמטר את id – מחרוזת המייצגת מספר זהות של תלמיד/ה. הפעולה מחזירה "אמת" אם במערך students קיים ערך זהה ל-id. אחרת יוחזר "שקר". אין צורך לממש את הפעולה. להלן חתימת הפעולה:

```
public bool InCapsule(string id)
```

כתוב פעולה חיצונית **AtLeastTwo** המקבלת כפרמטרים את caps – מערך קפסולות ואת ids – מערך מחרוזות המייצגות מספרי זהות של תלמידים. הפעולה תמנה ותחזיר כמה מספרי זהות במערך ids מופיעים בלפחות שתי קפסולות במערך caps. לצורך הפתרון יש להשתמש בפעולה **InCapsule**.

הנח כי במערך caps אין הפניות ריקות. הנח בנוסף כי במערך ids – אין הפניות ריקות וכי המחרוזות שונות זו מזו.

שאלה 3 – 15 נקודות

באפליקציית טיק טוק אפשר לצפות בסרטונים. "סרטון" (Clip) מאופיין על ידי משך בשניות (seconds, מטיפוס שלם), תגובות (talkbacks, מערך של 15 תגובות) ומונה תגובות בפועל (count, מטיפוס שלם שערכו בין 0 ל-15). הנח כי לתכונות seconds ו-count קיימות פעולות מאחזרות (Get).

"תגובה" (Talkback) מאופיינת על ידי מזהה כותב התגובה (id, מטיפוס שלם) וטקסט התגובה (text, מטיפוס מחרוזת). הנח כי לתכונות id ו-text קיימות פעולות מאחזרות (Get). הנח כי באפליקציה לכל כותב תגובה id שונה.

א. (2 נק') כתוב את כותרת המחלקה "סרטון" ואת תכונותיה.

ב. (4 נק') כתוב את הפעולה הבונה של המחלקה "סרטון" שמקבלת כפרמטר את seconds – מספר שלם המייצג את משך הסרטון. הפעולה תאתחל את מערך התגובות (talkbacks) ל-15 הפניות ריקות (null) ותציב 0 במשתנה התכונה count.

ג. (9 נק') במחלקה "סרטון" כתוב את הפעולה הבוליאנית **AddTalkback** המקבלת כפרמטר את tb – מטיפוס "תגובה". אם במערך התגובות (talkbacks) יש מקום ואין בו תגובה אחרת שנכתבה על ידי כותב התגובה tb, הפעולה תוסיף את tb למערך ותחזיר "אמת". אם לא מתקיימים התנאים להוספת tb למערך התגובות, הפעולה תחזיר "שקר".

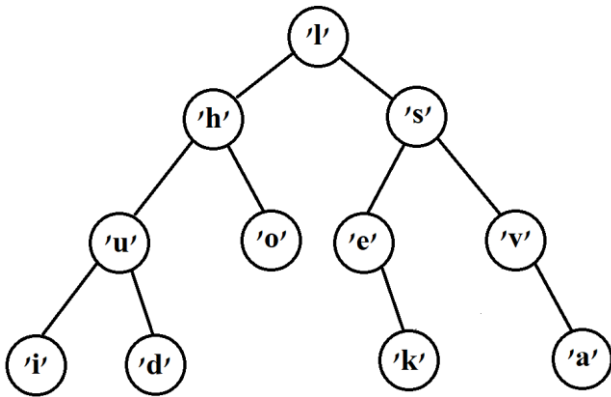
פרק שני (50 נקודות)

בפרק זה עליך לענות על שתיים מבין השאלות 4, 5 ו-6

שאלה 4 – 25 נקודות

א. (22 נק') כתוב את הפעולה החיצונית *ToStr* המקבלת כפרמטר את *t* - עץ לא ריק של תווי אותיות קטנות באנגלית. הפעולה **תיצור ותחזיר** מחזרת המכילה את תווי העלים של העץ, כך שהתו הראשון במחרוזת יהיה ערך העלה השמאלי ביותר בעץ, התו השני במחרוזת יהיה ערך העלה השני השמאלי בעץ, וכך הלאה... עד שהתו האחרון במחרוזת יהיה ערך העלה הימני ביותר בעץ.

למשל, עבור העץ הבא, הפעולה תחזיר את המחרוזת "idoka"



ב. (3 נק') כתוב מהי סיבוכיות הפעולה *ToStr*. נמק תשובתך.

שאלה 5 – 25 נקודות

נתון ממשק המחלקה *VLst*:

class VLst	
Node<int> vals – רשימת ערכים לא שליליים	תכונות
public VLst () { this.vals = null; }	פעולה בונה (כולל מימוש)
public int SumVals()	מחזירה את סכום הערכים ברשימה vals. במידה והרשימה ריקה יוחזר 0. סיבוכיות הפעולה $O(n)$. אין צורך לממש את הפעולה.

- א. (6 נק') הוסף למחלקה *VLst* את הפעולה *AddVal* המקבלת כפרמטר את *x* - מספר שלם לא שלילי. הפעולה תוסיף את *x* בסוף הרשימה vals.
- ב. (16 נק') כתוב פעולה **חיצונית** בשם *Update* המקבלת כפרמטר את *s* - מחסנית לא ריקה של עצמים מטיפוס *VLst*. הפעולה תוסיף בסוף הרשימה של כל עצם במחסנית את סכום רשימת הערכים של העצם הנמצא **מעליו** במחסנית. לצורך הפתרון, יש להשתמש בפעולות *SumVals* ו- *AddVal*. להלן דוגמה למצב העצמים של מחסנית כלשהי לפני זימון הפעולה *Update* ובסיום הפעולה.

לפני זימון הפעולה פתח המחסנית (Top)	בסיום הפעולה פתח המחסנית (Top)
{ vals: 3->5->>null }	{ vals: 3->5->>null }
{ vals: 9->0->1->8->>null }	{ vals: 9->0->1->>null }
{ vals: 10->>null }	{ vals: null }
{ vals: 19->4->2->0->>null }	{ vals: 19->4->2->>null }

ג. (3 נק') כתוב מהי סיבוכיות הפעולה *Update*. נמק תשובתך.

א. (10 נק') נתונה הפעולה **Sod** המקבלת שני פרמטרים: x - מטיפוס שלם שערכו חיובי גדול מ-0, ו- e - מטיפוס בוליאני.

```
public static int Sod(int x, bool e)
{
    int r = 0, m = 10;

    if ((x % 2 == 0) == e)
        r = x % 10;
    else
        m = 1;

    if (x > 10)
        r = Sod(x / 10, e) * m + r;

    return r;
}
```

בצע מעקב אחר הזימון הבא של הפעולה **Sod**, וציין בנוסף מה יהיה הפלט שיוצג:

```
int num1 = Sod(2741, true);
Console.WriteLine(num1);
```

מומלץ לשרטט את המעקב בדף נפרד וחדש

ב. (1 נק') כתוב מה יהיה הפלט שיוצג בזימון הבא של הפעולה **Sod**. אין צורך לבצע מעקב אלא לציין את הפלט בלבד.

```
int num2 = Sod(2741, false);
Console.WriteLine(num2);
```

ג. (2 נק') כתוב את טענת היציאה של הפעולה **Sod** (כלומר, הסבר בקצרה מה הפעולה מבצעת).

ד. (10 נק') נתונה הפעולה **Mystery** המזמנת את הפעולה **Sod** שלעיל.

הנח כי בזימון הפעולה **תמיד** אורכו של המערך arr זוגי ובכל התאים ממחציתו ועד סופו הערך הוא 0.

הנח בנוסף כי בזימון הפעולה **תמיד** ערכו של הפרמטר i הוא 0 וערכו של הפרמטר k הוא אורך המערך arr פחות 1.

```
public static void Mystery(int[] arr, int i, int k)
{
    if (i < k)
    {
        arr[k] = Sod(arr[i], false);
        arr[i] = Sod(arr[i], true);
        Mystery(arr, i + 1, k - 1);
    }
}
```

בצע מעקב אחר הזימון בהמשך של הפעולה **Mystery** עבור המערך arr .

בכל שלב של המעקב יש לציין את ערכי המשתנים/פרמטרים ואת הערכים במערך.

אין צורך לכלול את המתרחש בתוך הפעולה **Sod**.

```
//תאים:      0      1      2      3
int[] arr = {756, 1234, 0, 0};
Mystery(arr, 0, arr.Length - 1);
```

מומלץ לשרטט את המעקב בדף נפרד וחדש

ה. (2 נק') כתוב את טענת היציאה של הפעולה **Mystery** (כלומר, הסבר בקצרה מה הפעולה מבצעת).

פרק שלישי (25 נקודות)

בפרק זה עליך לענות על שאלה אחת מבין השאלות 7 ו-8

שאלה 7 – 25 נקודות

הסעיפים בשאלה זו קשורים זה לזה ומתייחסים לחלקים שונים של אותה תכנית.

א. (10 נק') להלן החלק הראשון של התכנית. כל ההוראות בחלק זה תקינות.

```
B b1 = new A();
B b2 = new D();
D d1 = new E();
B b3 = new C();
A a1 = (A)b2;
```

ידוע כי כתיבת ההוראה הבאה תגרום לשגיאת הידור (קומפילציה):

```
A a2 = new C();
```

בהתבסס על ההוראות התקינות ועל ההוראה השגויה, שרטט תרשים המציג את קשרי ההורשה בין המחלקות.

ב. (8 נק') להלן החלק השני של התכנית. ההוראות בחלק זה תקינות ומופיעות לאחר אלו שבחלק הראשון של התכנית.

בהערות מצוין הפלט המוצג על המסך בעקבות ביצוע ההוראה, למעט בהוראה המסומנת בסולמית (#).

```
((D)b2).Where(); //output - "I am not there"
d1.Where();      //output - "Now I am here"
((A)b1).Where(); //output - "Here I am"
a1.Where();      //#
```

ידוע כי כתיבת ההוראה הבאה תגרום לשגיאת הידור (קומפילציה):

```
b1.Where();
```

בהתבסס על ההוראות התקינות ועל ההוראה השגויה:

1. באילו מחלקות וכיצד תוגדר הפעולה *Where*? כתוב את כותרת הפעולה בכל מחלקה.

2. כתוב מה יהיה הפלט של ההוראה המסומנת בסולמית (#)

ג. (7 נק') להלן חלק נוסף של התכנית. גם ההוראות בחלק זה תקינות ומופיעות לאחר אלו שבחלק הראשון של התכנית.

ידוע כי אורך המערך arr הוא 4 (ואין בו הפניות ריקות) וכי הפעולה F1 מוגדרת באחת מבין המחלקות A, B, C, D, E.

```
d1.F1();
for(int i = 0; i < arr.Length; i++)
{
    if(arr[i] is A)
        arr[i].F1();
}
```

עוד ידוע כי כתיבת ההוראה הבאה תגרום לשגיאת הידור (קומפילציה):

```
a1.F1();
```

לגבי כל אחת מההוראות הבאות קבע האם היא יכולה, או לא יכולה, להתאים להגדרת המערך arr. נמק כל קביעה.

1. `object[] arr = new object[4];`

2. `A[] arr = new A[4];`

3. `B[] arr = new B[4];`

4. `C[] arr = new C[4];`

5. `D[] arr = new D[4];`

6. `E[] arr = new E[4];`

לכבוד יום הולדתו של רועי חבריו רוצים לרכוש משחק ומתלבטים בין האפשרויות הבאות:

- **משחק (Game)** - מאופיין על ידי שם (name, מטיפוס מחרוזת) ומספר משתתפים (count, מטיפוס שלם)
- **משחק קופסה (BoxGame)** - הוא משחק המאופיין בנוסף על ידי סוג (kind - מטיפוס שלם. 1 - קלפים, 2 - לוח, 3 - פאזל)
- **משחק מחשב (CompGame)** - הוא משחק המאופיין בנוסף על ידי דירוג פופולאריות (rank - שלם בין 1 ל-5)
- **משחק אינטרנטי (WebGame)** - הוא משחק מחשב המאופיין בנוסף על ידי כתובת אתר האינטרנט (url - מטיפוס מחרוזת)
- **אפליקציה (AppGame)** - היא משחק מחשב להתקנה על הטלפון ומאופיינת בנוסף על ידי גודל הקובץ (size - מטיפוס שלם)

א. (5 נק') שרטט תרשים עץ הורשה המציג את הקשרים בין המחלקות.

לכל מחלקה בתרשים יופיעו התכונות, פעולה בונה מלאה ופעולה מאחזרת (Get) לכל אחת מתכונות.

טיפ: בפעולה הבונה השם של כל פרמטר יכול להיות האות הראשונה של התכונה המתאימה (למשל n עבור name).

ב. (10 נק') חבריו של רועי חושבים שהוא יעדיף את אחד המשחקים הבאים:

- **משחק קופסה של קלפים** לשני משתתפים לפחות.
- **משחק אינטרנטי אן אפליקציה** לשני משתתפים לפחות ובעלת דירוג פופולאריות גבוה מ-3.

כתוב את הפעולה החיצונית ChooseGame המקבלת כפרמטר את games – מערך של משחקים.

הפעולה **תחזיר** את המשחק הראשון במערך games שמתאים להיות מתנה לרועי.

במידה ואין במערך משחק מתאים, הפעולה **תחזיר** null. הנח כי במערך games אין הפניות ריקות (null).

ג. (10 נק') בצד הימני של הטבלה ניתן לראות כיצד הפעולה Equals נדרסה במחלקה Game ובמחלקה BoxGame.

בצד השמאלי של הטבלה נתונה תכנית שעוברת הידור ורצה בהצלחה.

שרטט את העצמים במערך arr ובמערך copy בסיום ריצת התכנית.

<pre>Game[] arr = new Game[4]; arr[0] = new BoxGame("G1", 3, 2); arr[1] = new Game("G1", 3); arr[2] = new BoxGame("G1", 3, 3); arr[3] = arr[1]; int k = 0; Game[] copy = new Game[arr.Length]; for(int i = 0; i < arr.Length; i++) { copy[k] = arr[i]; for(int j = 0; j < k; j++) { if(copy[k].Equals(copy[j])) { copy[k] = null; j = k; k--; } } k++; }</pre>	במחלקה Game <pre>public override bool Equals(object obj) { if(obj is Game) { Game g = (Game)obj; if(name == g.name && count == g.count) return true; } return false; }</pre>
	במחלקה BoxGame <pre>public override bool Equals(object obj) { if(obj is BoxGame) { bool b = base.Equals(obj); BoxGame bg = (BoxGame)obj; return b && (kind == bg.kind); } return false; }</pre>